https://github.com/Et1enneee/comp3500-logistics

COMP 3500SEF · Final Report

Warehouse Logistics Management System
Group 42: Tang Yi Jie, Yin Pei Xu, Tang Chao, Yang Ting Zhang, Wu Peng Bo, Chung Ngon Man, Li Mu Qiu.

1. Scope

The goals are traceable order decisions and a role-separated workflow. Customers order and track; staff confirm, pick, ship and assign; drivers simulate assigned deliveries; administrators report. The local application serves one warehouse using Next.js, TypeScript, Prisma and SQLite. Real GPS, barcode scanning, recipient telephone collection, payment and carrier integration are outside scope. Operational constraints address deployment and data integrity.

2. Development process

Plan-driven incremental development defines each increment by its trigger and verified outcome; integration and continuous integration provide feedback. Product or specification changes form increments; evidence production belongs to hardening. The records establish development, not external delivery.

Table A. Increment, trigger and outcome

Increment and date	Trigger	Outcome
Planning, 9 and 11 Oct 2026	Project selection	Responsibilities and contracts recorded
Test-led review, 17 Oct 2026	Assess order and stock functions	Defects recorded; drove confirmation
Confirmation, 26–28 Oct 2026	Failed endpoint and missing ledger	Transaction verified
Integrated application, 1 Nov 2026	Integrate foundation	Customer, staff and admin workflows
Driver and privacy, 2 Nov 2026	Delivery and data minimisation	Assigned tasks and privacy checks
Documentation hardening, 10–11 Nov 2026	Model and evidence review	Diagrams and traceability aligned
Export simplification, 11 Nov 2026	Scripted walkthrough finding (issue #15)	Selected-date export in three actions; regression passed

Git manages versions, the workflow integrates changes, and the issue tracker records problems. Increment evidence.

3. Requirements

The software requirements specification is the specification. Functional requirements cover FR-01 order creation; FR-02 state changes; FR-03 numbering and tracking; FR-04 inventory and alerts; FR-05 notifications; FR-06 reports; FR-07 access control; FR-08 simulated delivery. Non-functional requirements cover NFR-01 performance; NFR-02 integration-point availability; NFR-03 least privilege; NFR-04 scripted operation effort; NFR-05 process quality.

Table B. Risk coverage (risk matrix)

Risk	Addressed by	Treatment status
RK-01	Increment register	Managed — scheduling plan
RK-02	FR-02, NFR-05	Mitigated — transition tests
RK-03	Priority register	Managed — scope priorities
RK-04	NFR-05	Managed — CI workflow
RK-05	FR-01/04/06, NFR-05	Managed — seed regression test
RK-06	Risk plan	Managed — assessment schedule
RK-07	NFR-05 support	Managed — setup guide
RK-08	Risk plan	Managed — report and video plan

Mitigated records a mitigation; managed records a mechanism while the historical register remains open.

4. System models

Layered architecture separates pages, routes and authentication, business handlers, and persistence. Transactions preserve consistency without splitting this local warehouse into separate services. State and stock guards and transaction atomicity enforce hard rules.

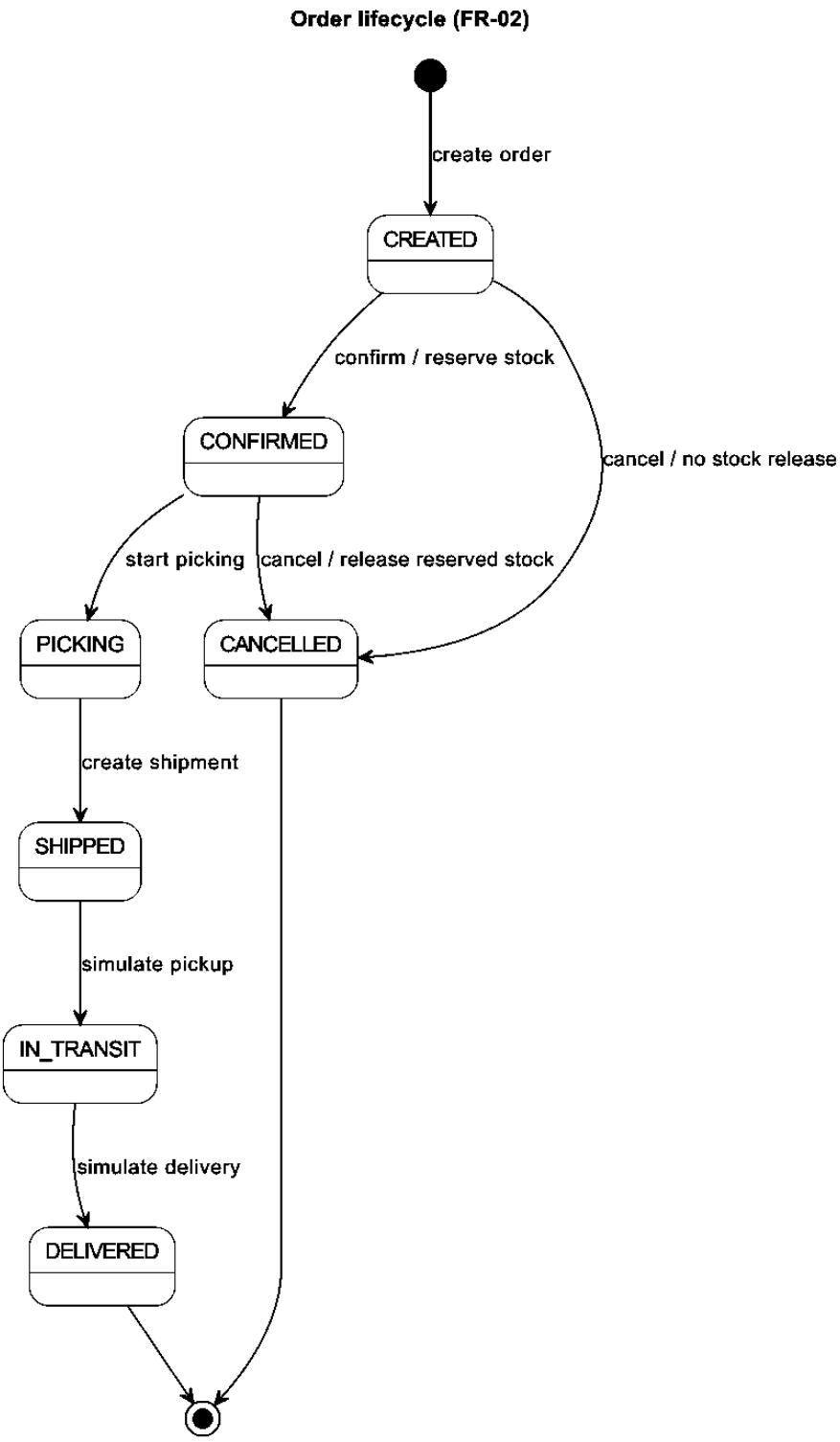


Figure 1. Order lifecycle: valid transitions and effects. Source.

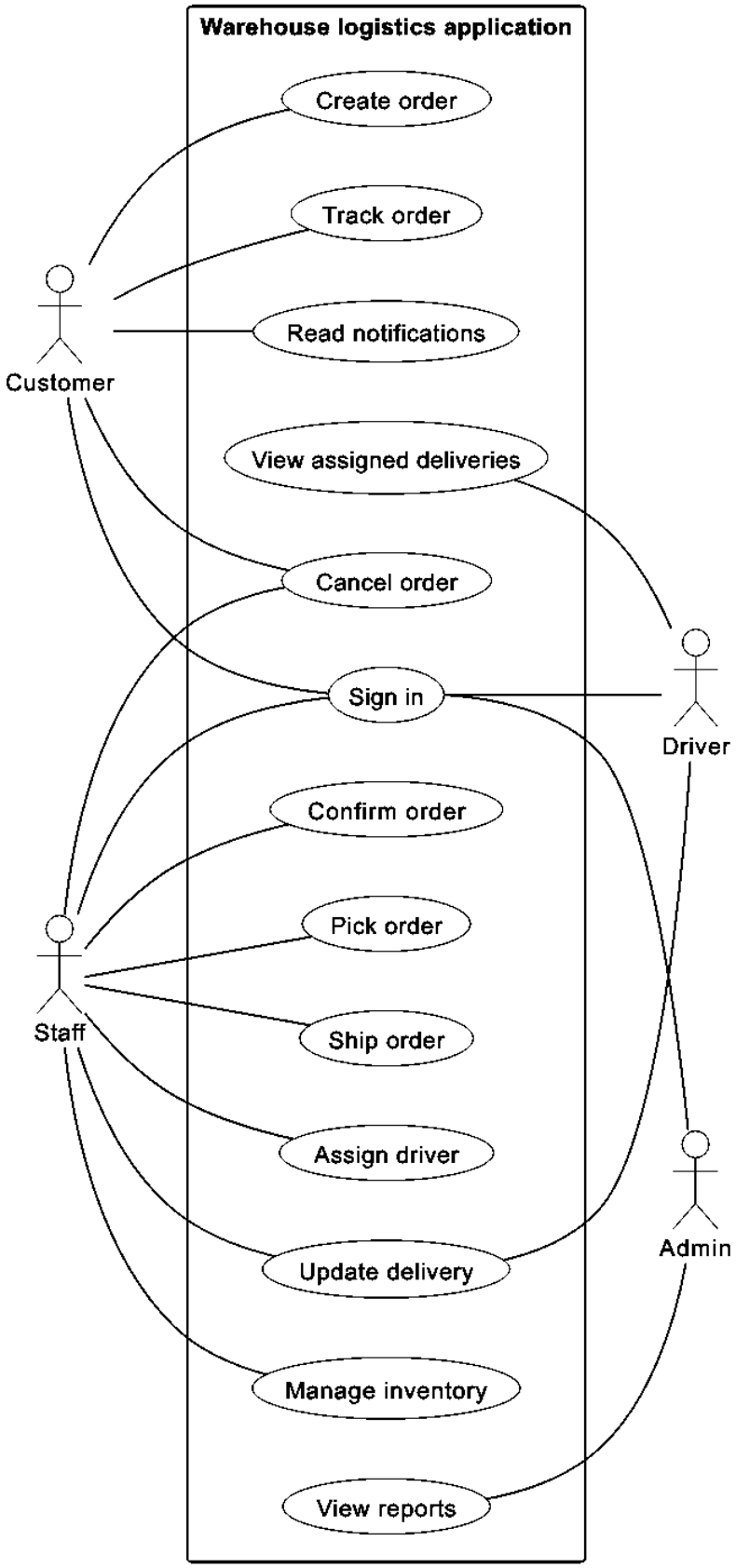


Figure 2. Use cases associate external roles with application functions. Source.

An order has 0..* items in the schema, while creation requires at least one; an item references one product. An order has 0..1 shipment; a product has 0..* inventory logs.

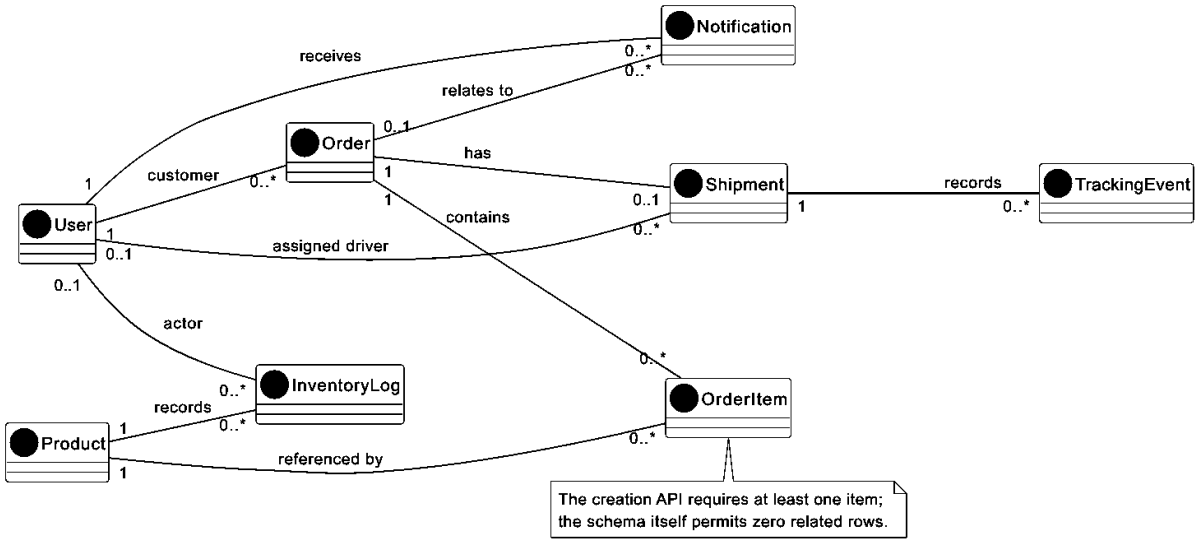


Figure 3. Entity relationships and multiplicities. Source.

5. Implementation

The confirmation guard (order-api.ts:250–253) requires CREATED; the stock guard (order-api.ts:276–280) requires sufficient quantity. The serializable transaction groups state, stock, ledger and notification writes. Staff cancellation requires CONFIRMED. A driver is assigned only to SHIPPED orders; the assignment remains after delivery.

6. Verification and validation

Verification checks the specification; validation assesses fitness for real users. The recorded run passed 38 unit, 140 integration, 2 delivery and 26 browser checks; logs and CI provide evidence. Repeated confirmation and insufficient stock return HTTP 409 with unchanged state and stock. In a pre-export-fix local-loopback measurement, API p95 was 3.2–13.6 ms and first-contentful-paint p95 was 64–96 ms, against NFR-01 targets of at most 2 s and 3 s respectively (measurement). The revised availability criterion records 40 successful build–test–browser integration points (observations).

7. Evolution

The early defect record led to the confirmation transaction and its retest. The post-presentation audit led to corrected models and renewed tests. The walkthrough exposed a four-action administrator task; direct export now takes three actions including date entry, or one click from a prepared screen (regression).

8. Validation

The automated walkthrough counts at most three scripted actions for each of eleven selected operations from prepared screens (excluding login, navigation and recovery). The human protocol defines role tasks, completion, errors, help requests and time, with acceptance criteria defined before each run. Participant observation is planned for 17–24 November.

9. Feedback and reconciliation

Table C. Feedback to evidence

Feedback	Disposition	Evidence
Diagrams	Corrected state and use-case models	Audit
FR-04	Stock transition effects belong to FR-02	Traceability
Increments	Defined triggers and outcomes	Register
Presentation p95	Evidence file governs; the presentation figure has no tracked measurement in any baseline	Measurement
Admin task	Direct export re-tested; participant observation planned (§8)	Issue #15
Unit count	Two soak-summary tests increased the count from 36 to 38	Test history
NFR scope	Criteria revised for local deployment	SRS revision log

The matrices and increment records connect decisions to implementation and evidence.

10. Evidence

Traceability · Specification · Evidence directory · Baseline record.

Full evidence trail: docs/final-report.md in the repository (evidence baseline `7f161031313ffa0cf43efb1fdedf71c220ef220f`).